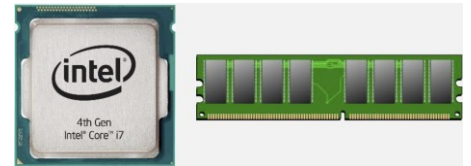

FUNCTIONAL PROGRAMMING

SENG 426

Introduction

- The design of the imperative languages is based directly on the *von Neumann architecture*

- Efficiency is the primary concern, rather than the suitability of the language for software development



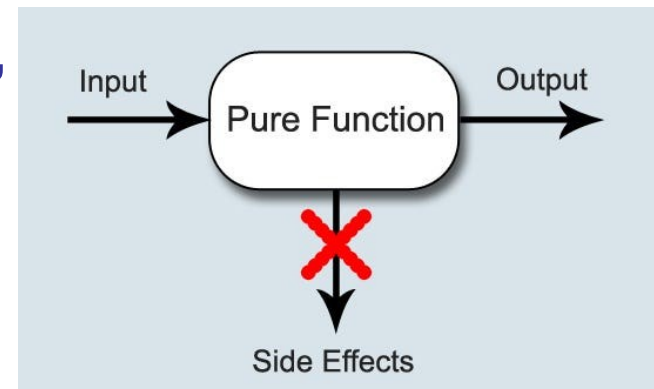
- The design of the functional languages is based on *mathematical functions*

- A solid theoretical basis that is also closer to the user, but relatively unconcerned with the architecture of the machines on which programs will run

$$f(x)$$

Fundamentals of Functional Programming Languages

- The objective of the design of a FPL is to **mimic mathematical functions** to the greatest extent possible
- The basic process of computation is fundamentally different in a FPL than in an imperative language
 - In an imperative language, operations are done and the results are stored in **variables** for later use
 - **Management of variables** is a constant concern and source of complexity for imperative programming
- In an FPL, **variables are not necessary**, as is the case in mathematics
- ***Referential Transparency*** – In an FPL, the evaluation of a function always produces the same result given the same parameters



Mathematical Functions

- A mathematical function is a *mapping* of members of one set, called the *domain set*, to another set, called the *range set*
- A *lambda expression* specifies the parameter(s) and the mapping of a function in the following form

$$\lambda (x) \quad x * x * x$$

for the function

$$\text{cube}(x) = x * x * x$$


Lambda Expressions

- Lambda expressions describe **nameless functions**
- Lambda expressions are **applied to parameter(s)** by placing the parameter(s) after the expression

e.g., $(\lambda (x) \ x \ * \ x \ * \ x) \ (2)$

which evaluates to 8

Functional Forms

A **higher-order function**, or *functional form*, is one that either takes **functions as parameters** or **yields a function** as its result, or both

$$f(g(x))$$

$$f(g(x)) \rightarrow h(y)$$

Function Composition

- A functional form that **takes two functions as parameters** and **yields a function** whose value is the first actual parameter function applied to the application of the second

Form: $h \equiv f \circ g$

which means

$$h(x) \equiv f(g(x))$$

For

$$f(x) \equiv x + 2$$

and

$$g(x) \equiv 3 * x$$

$$h \equiv f \circ g$$

yields

$$(3 * x) + 2$$

Apply-to-all (map)

- A functional form that takes a **single function as a parameter** and yields a list of values obtained by **applying** the given function to each element of a list of parameters

Form: α

For $h(x) \equiv x * x$

$\alpha(h, (2, 3, 4))$ yields $(4, 9, 16)$

Lisp Data Types and Structures

- *Data object types*: originally only atoms and lists
- *List form*: parenthesized collections of sublists and/or atoms

e.g.,

(A B (C D) E)

- Originally, **Lisp** was a typeless language
- Lisp lists are stored internally as **single-linked lists**

Lisp Interpretation

- Lambda notation is used to specify functions and function definitions. **Function applications** and **data** have the **same form**.

e.g., If the list `(A B C)` is interpreted as **data** it is a simple list of three atoms, A, B, and C
If it is interpreted as a **function application**, it means that the function named A is applied to the two parameters, B and C

List Types (continued)

- List Operations in Scheme (Lisp dialect)

- CAR returns the first element of its list parameter

```
(car ' (A B C) ) returns A
```

- CDR returns the remainder of its list parameter after the first element has been removed

```
(cdr ' (A B C) ) returns (B C)
```

- CONS puts its first parameter into its second parameter, a list, to make a new list

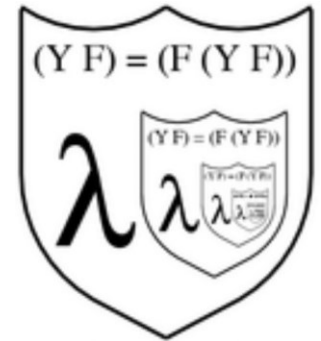
```
(cons 'A ' (B C) ) returns (A B C)
```

- LIST returns a new list of its parameters

```
(list 'A 'B ' (C D) ) returns (A B (C D) )
```

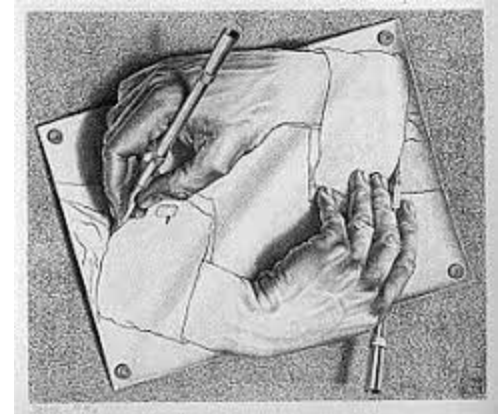
Origins of Scheme

- A mid-1970s **dialect of Lisp**, designed to be a cleaner, more modern, and simpler version than the contemporary dialects of Lisp
- Uses only **static scoping**
- Functions are **first-class entities**
 - They can be the values of expressions and elements of lists
 - They can be assigned to variables, passed as parameters, and returned from functions



The Scheme Interpreter

- In interactive mode, the Scheme interpreter is an infinite **read-evaluate-print loop** (REPL)
 - This form of interpreter is also used by Python and Ruby
- **Expressions** are interpreted by the function `EVAL`
- **Literals** evaluate to themselves



Primitive Function Evaluation

- Parameters are evaluated, in **no particular order**
- The **values** of the parameters are substituted into the function body
- The function body is evaluated
- The value of the **last expression** in the body is the (return) value of the function

```
(lambda (p1 p2)
  ...
  (+ (+ p1 p2) (/ p1 p2) )
)
```

Primitive Functions & LAMBDA Expressions

- Primitive Arithmetic Functions: +, −, *, /, ABS, SQRT, REMAINDER, MIN, MAX

e.g., `(+ 5 2)` yields 7

- Lambda Expressions

- Form is based on λ notation

e.g., `(lambda (x) (* x x))`

`x` is called a **bound variable**

- Lambda expressions can be applied to parameters

e.g., `((lambda (x) (* x x)) 7)`

- LAMBDA expressions can have any number of parameters

`(lambda (a b x) (+ (* a x x) (* b x)))`

Special Form Function: DEFINE

DEFINE – Two forms:

1. To bind a **symbol** to an **expression**

e.g., `(define pi 3.141593)`

Example use: `(define two_pi (* 2 pi))`

These symbols are not variables – they are like the names bound by Java's `final` declarations

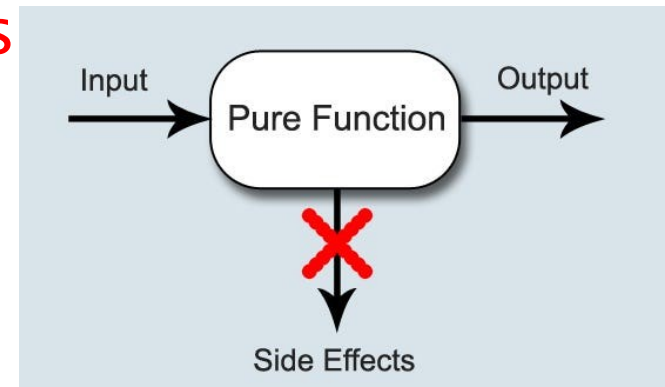
2. To bind names to lambda expressions (`LAMBDA` is implicit)

e.g., `(define (square x) (* x x))`

Example use: `(square 5)`

Output Functions

- Usually not needed, because the interpreter always displays the result of a function evaluated at the **top level** (not nested)
- Scheme has `DISPLAY`, which is similar to the `printf` function of C
- **Note:** explicit input and output are not part of the pure functional programming model, because **input operations** change the state of the program and **output operations** are side effects



Numeric Predicate Functions

- $\#T$ (or $\#t$) is true and $\#F$ (or $\#f$) is false (sometimes $()$ is used for false)
- `=, <>, >, <, >=, <=`
- `even?, odd?, zero?, negative?`
- The `not` function inverts the logic of a Boolean expression

Control Flow

- Selection– the special form, `IF`
(`IF` predicate then_exp else_exp)

```
(if (<> count 0)
    (/ sum count)
)
```

- The `COND` function:

```
(define (leap? year)
  (cond
    ((zero? (modulo year 400)) #T)
    ((zero? (modulo year 100)) #F)
    (else (zero? (modulo year 4))))
))
```

List Functions

- **QUOTE** – takes one parameter; returns the parameter without evaluation
 - QUOTE is required because the Scheme interpreter, named EVAL, always evaluates parameters to function applications before applying the function. QUOTE is used to avoid parameter evaluation when it is not appropriate
 - QUOTE can be abbreviated with the apostrophe (single quote) prefix operator

' (A B)

is equivalent to

(quote (A B))

List Functions (continued)

- Examples:

<code>(car ' ((A B) C D))</code>	returns	<code>(A B)</code>
<code>(car 'A)</code>	returns	error
<code>(cdr ' ((A B) C D))</code>	returns	<code>(C D)</code>
<code>(cdr 'A)</code>	returns	error
<code>(cdr ' (A))</code>	returns	<code>()</code>
<code>(cons ' () ' (A B))</code>	returns	<code>(() A B)</code>
<code>(cons ' (A B) ' (C D))</code>	returns	<code>((A B) C D)</code>
<code>(cons 'A 'B)</code>	returns	<code>(A . B)</code>

List Functions (continued)

LIST is a function for building a list from any number of parameters

```
(list 'apple 'orange 'grape)
```

 returns

```
(apple orange grape)
```

Predicate Function: EQ?

EQ? takes two expressions as parameters (usually two **atoms**); it returns **#T** if both parameters have the same **pointer** value; otherwise **#F**

```
(eq? 'A 'A) yields #T
```

```
(eq? 'A 'B) yields #F
```

```
(eq? 'A '(A B)) yields #F
```

```
(eq? '(A B) '(A B)) yields #T or #F (avoid using)
```

```
(eq? 3.4 (+ 3 0.4)) yields #T or #F (avoid using)
```

Predicate Function: `EQV?`

`EQV?` is like `EQ?`, except that it works for both symbolic and numeric atoms; it is a **value comparison**, not a pointer comparison

```
(eqv? 3 3)
```

yields `#T`

```
(eqv? 'A 3)
```

yields `#F`

```
(eqv? 3.4 (+ 3 0.4))
```

yields `#T`

```
(eqv? 3.0 3)
```

yields `#F`

(floats and integers are different)

Predicate Functions: LIST? and NULL?

- **LIST?** takes one parameter; it returns #T if the parameter is a list; otherwise #F

`(list? '())` yields #T

`(list? 2)` yields #F

- **NULL?** takes one parameter; it returns #T if the parameter is the empty list; otherwise #F

`(null? '(()))` yields #F

`(null? '())` yields #T

Example Scheme Function: `member`

member takes an atom and a simple list;
returns `#T` if the atom is in the list; `#F`
otherwise

```
(define (member atm a_list)
  (cond
    ((null? a_list) #F)
    ((eq? atm (car a_list)) #T)
    (else (member atm (cdr a_list))))
)
```

Example Scheme Function: equalsimp

equalsimp takes two simple lists as parameters;
returns **#T** if the two simple lists are equal; **#F**
otherwise

```
(define (equalsimp list1 list2)
  (cond
    ((null? list1) (null? list2))
    ((null? list2) #F)
    ((eq? (car list1) (car list2))
     (equalsimp(cdr list1) (cdr list2)))
    (else #F)
  )
)
```

Example Scheme Function: `equal`

`equal` takes two general lists as parameters; returns `#T` if the two lists are equal; `#F` otherwise

```
(define (equal list1 list2)
  (cond
    ((not (list? list1)) (eq? list1 list2))
    ((not (list? list2)) #F)
    ((null? list1) (null? list2))
    ((null? list2) #F)
    ((equal (car list1) (car list2))
     (equal (cdr list1) (cdr list2)))
    (else #F)
  )
)
```


Example Scheme Function: LET

- **LET** is actually shorthand for a LAMBDA expression applied to a parameter

```
(let ((alpha 7)) (* 5 alpha))
```

is same as:

```
((lambda (alpha) (* 5 alpha)) 7)
```

LET Example

```
(define (quadratic_roots a b c)

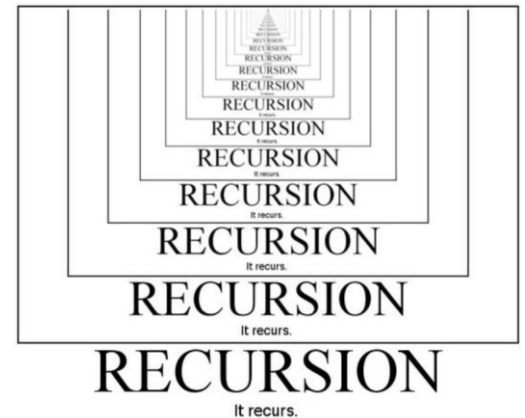
  (let ((root_part_over_2a
         (/ (sqrt (- (* b b) (* 4 a c))) (* 2 a))
         (minus_b_over_2a
          (/ (- 0 b) (* 2 a))))

    (list (+ minus_b_over_2a
              root_part_over_2a)
           (- minus_b_over_2a
              root_part_over_2a))

  )
)
```

Tail Recursion in Scheme

- Definition: A function is *tail recursive* if its recursive call is the last operation in the function
- A tail recursive function can be automatically converted by a compiler **to use iteration**, making it faster
- Scheme language definition requires that Scheme language systems convert all tail recursive functions to use iteration



Tail Recursion in Scheme – continued

Example of rewriting a function to make it **tail recursive**, using helper a function

Original:

```
(define (factorial n)
  (if (<= n 0)
      1
      (* n (factorial (- n 1)))
  ))
```

Tail recursive:

```
(define (helper n accumulated)
  (if (<= n 0)
      accumulated
      (helper (- n 1) (* n accumulated))))
(define (factorial n)
  (helper n 1))
```

Functional Form – Composition

- Composition

- If h is the composition of f and g , $h(x) = f(g(x))$

```
(define (g x) (* 3 x))
```

```
(define (f x) (+ 2 x))
```

```
(define (h x) (+ 2 (* 3 x)))
```

 (The composition)

- In Scheme, the functional composition function `compose` can be written:

```
(define (compose f g) (lambda (x) (f (g x))))
```

```
((compose car cdr) '((a b) c d))
```

 yields `c`

```
(define (third a_list)  
  ((compose car (compose cdr cdr)) a_list))
```

is equivalent to `CADDR`

Functional Form – Apply-to-All

- Apply to All – one form in Scheme is `map`
 - Applies the given function to all elements of the given list;

```
(define (map fun a_list)
  (cond
    ((null? a_list) '())
    (else (cons (fun (car a_list))
                  (map fun (cdr a_list)))))
  )
)
```

```
(map (lambda (num) (* num num num)) '(3 4 2 6))
(27 64 8 216)
```

yields

Functions That Build Code

- It is possible in Scheme to define a function that **builds Scheme code** and requests its interpretation
- This is possible because the interpreter is a user-available function, `EVAL`

Adding a List of Numbers

```
(define (adder a_list)
  (cond
    ((null? a_list) 0)
    (else (eval (cons '+ a_list))))
)
```

- The parameter is a list of numbers to be added; `adder` inserts a `+` operator and evaluates the resulting list
 - Use `CONS` to insert the atom `+` into the list of numbers.
 - Be sure that `+` is quoted to prevent evaluation
 - Submit the new list to `EVAL` for evaluation

Backquote and Quote

Backquote operator (`)

- Similar to the Scheme's `QUOTE`, except that some parts of the parameter can be unquoted by preceding them with **commas**

``(a (* 3 4) c)` evaluates to `(a (* 3 4) c)`

``(a , (* 3 4) c)` evaluates to `(a 12 c)`

Common Lisp

- A combination of many of the features of the popular dialects of Lisp around in the early 1980s
- A **large** and **complex** language--the opposite of Scheme
- Features include:
 - records
 - arrays
 - complex numbers
 - character strings
 - powerful I/O capabilities
 - packages with access control
 - iterative control statements

Haskell

- Static scoped, strongly typed, type inferencing, pattern matching
- *Purely functional* (e.g., no variables, no assignment statements, and no side effects of any kind)

Syntax:

```
factorial :: Integer -> Integer
factorial 0 = 1
factorial n = n * factorial (n-1)
```

```
fibonacci :: Integer -> Integer
fibonacci 0 = 1
fibonacci 1 = 1
fibonacci n = fibonacci (n-1) + fibonacci (n-2)
```


Haskell Lists

- **List notation:** Put elements in brackets

```
directions = ["north", "south", "east", "west"]
```

- **Length:** #

```
#directions is 4
```

- **Arithmetic series with the .. operator**

```
[2, 4..10] is [2, 4, 6, 8, 10]
```

- **Catenation is with ++**

```
[1, 3] ++ [5, 7] results in [1, 3, 5, 7]
```

- **CONS via the colon operator**

```
1:[3, 5, 7] results in [1, 3, 5, 7]
```

Haskell (continued)

- Pattern Parameters

```
product [] = 1  
product (a:x) = a * product x
```

- Factorial:

```
fact n = product [1..n]
```

- List Comprehensions

```
[n * n * n | n <- [1..50]]
```

Quicksort

```
sort [] = []
sort (h:t) =
    sort [b | b <- t, b <= h]
    ++ [h] ++
    sort [b | b <- t, b > h]
```

Illustrates the concision of Haskell

Lazy Evaluation

- A language is *strict* if it requires all actual parameters to be fully evaluated
- A language is *nonstrict* if it does not have the strict requirement
- Nonstrict languages are more efficient and allow some interesting capabilities – *infinite lists*
- **Lazy evaluation** – Only compute those values that are necessary
- Positive numbers

```
positives = [0..]
```

- Determining if 16 is a square number

```
member [] b = False
member b (a:x)=(a == b) || member x b
squares = [n * n | n <- [0..]]
member squares 16
```

Member Revisited

- The member function could be written as:

```
member b [] = False
member b (a:x) = (a == b) || member b x
```

- However, this would only work if the parameter to squares was a perfect square; if not, it will keep generating them forever. The following version will always work:

```
member n (m:x)
  | m < n = member n x
  | m == n = True
  | otherwise = False
```

Support for Functional Programming in Primarily Imperative Languages

Support for functional programming is increasingly creeping into imperative languages

- Anonymous functions (lambda expressions)

- JavaScript: leave the name out of a function definition

```
(function (a) { ... } ) ()
```

- C#: `i => (i % 2) == 0` returns true or false depending on whether the parameter is even or odd)

- Python: `lambda a, b : 2 * a - b`

Support for Functional Programming in Primarily Imperative Languages (continued)

- Python supports the higher-order functions filter and map (often use lambda expressions as their first parameters)

```
map(lambda x : x ** 3, [2, 4, 6, 8])
```

returns [8, 64, 216, 512]

- Python supports partial function applications

```
from operator import add  
from functools import partial  
add5 = partial (add, 5)
```

use: add5(15) will correspond to add(5, 15)

Support for Functional Programming in Primarily Imperative Languages (continued)

Ruby Blocks

- Subprograms that are sent to methods, which makes the method a **higher-order subprogram**
- A block can be converted to a subprogram object with `lambda`

```
times = lambda {|a, b| a * b}
```

Use: `x = times.(3, 4)` (sets `x` to 12)

- Times can be curried with

```
times5 = times.curry.(5)
```

Use: `x5 = times5.(3)` (sets `x5` to 15)

Comparing Functional and Imperative Languages

- Imperative Languages:
 - Efficient execution
 - Complex semantics
 - Complex syntax
 - Concurrency is programmer designed
- Functional Languages:
 - Simple semantics
 - Simple syntax
 - Less efficient execution
 - Programs can automatically be made concurrent

Summary

- Functional programming languages use **function application**, conditional expressions, recursion, and functional forms to control program execution
- Lisp began as a **purely functional** language and later included imperative features
- **Scheme** is a relatively simple dialect of Lisp that uses static scoping exclusively
- **Common Lisp** is a large Lisp-based language
- Haskell is a **lazy functional language** supporting infinite lists and set comprehension.
- Some primarily imperative languages now incorporate some **support for functional programming**
- Purely functional languages have advantages over imperative alternatives, but still are **not very widely used**